Session 16 · Tuesday 20 October 2026 · Four problems. Everything starts at zero, and that is the whole difficulty.

Ticks, glitches and depth

The rule for every table on this sheet. Time runs in ticks. At each tick every gene looks at the values of its inputs **as they were at the previous tick** and updates. All genes update together. At $t = 0$ the cell has just been made, so **every gene is at 0** and the inputs are already at their given values.

The three kinds of glitch, named by what the output was supposed to be doing:

	THE OUTPUT SHOULD HAVE	WHAT IT DOES INSTEAD
static-0	stayed at 0	goes up and comes back
static-1	stayed at 1	goes down and comes back
dynamic	changed once	changes more than once on the way

One number for depth. A chain of n genes, each driving the next, with a step at the input and every protein sharing a lifetime τ:

$$t_{1/2}(n) \;=\; \text{time for the output to reach half its final value,} \qquad t_{1/2}(n) \;\approx\; \left(n - \tfrac{1}{3}\right)\tau.$$

1 — Fully worked: the classifier, $A = 1, B = 0$

The circuit from class. $R_1 = \text{NOT } B$. R_2 is one gene behind two promoters, one repressed by A and one by R_1, so $R_2 = \text{NAND}(A, R_1)$. $R_3 = \text{NOT } R_2$, and R_3 is the output. With $A = 1$ and $B = 0$ the table settles to $R_3 = 1$.

t	R_1	R_2	R_3
0	0	0	0
1	1	1	1
2	1	0	0
3	1	0	1
4	1	0	1

Read the last column: 0, 1, 0, 1, 1. The output was supposed to change once, from its reset value to 1. It changed three times. **That is a dynamic hazard**, and it is the case whose correct answer is 1 — the circuit arrives at the right answer having first given it, taken it back, and given it again.

Why does that step follow? — at $t = 1$ every gene turns on at once. R_2 is a NAND, which is ON unless both its repressors are present, and at $t = 0$ neither is. R_3 is a NOT, which is ON unless its repressor is present, and at $t = 0$ R_2 is not. An empty cell is a cell with every repressor gate already switched on.

2 — Compile XNOR, and count the depths

XNOR is 1 when A and B agree. Build it from NOT and NOR gates only. Write the network as a list of genes, each line saying what represses what.

_____

_____

_____

_____

_____

_____

Count gates along every route from an input to the output.

$$\textbf{longest path} = _____ \textbf{ genes}, \qquad \textbf{shortest path} = _____ \textbf{ genes}$$

Why does that step follow? — a different group will hand in a different network that computes the same table. What, if anything, makes one of them better?

---

3 — A different circuit

Three genes, two inputs, wired differently from item 1:

$$R_1 = \text{NOT } A, \qquad R_2 = \text{NOR}(B,\, R_1), \qquad \text{out} = \text{NOT } R_2.$$

Fill four ticks for each input case. Every gene starts at 0.

t	R_1	R_2	OUT		t	R_1	R_2	OUT
$A=0,\ B=0$					$A=0,\ B=1$			
0					0			
1					1			
2					2			
3					3			
4					4			
$A=1,\ B=0$					$A=1,\ B=1$			
0					0			
1					1			
2					2			
3					3			
4					4			

(a) What function does it settle to? Write the truth table.

A	B	out
0	0	__
0	1	__
1	0	__
1	1	__

(b) Which input cases glitch, and which kind is each?

(c) One sentence. Not "because it glitches." **What in the wiring let a wrong value out?**

4 — How late is a four-gene chain?

Four genes in a chain, every protein with the same lifetime τ, a step at the input at $t = 0$. Do not compute the sum; use the rule of thumb.

$$t_{1/2}(4) \approx _____\ \tau$$

How much later is that than the three-gene chain?

$$_____\ \tau$$

Why does that step follow? — would you have guessed that number before this session? Most people underestimate depth. What about the first stage makes it misleading?

---

5 — Stretch: two conditions, and neither is enough

A circuit is **fail-safe** if its output is 0 in a cell that has only just been made. It is **balanced** if every route from any input to the output has the same number of genes.

Item 3's circuit is neither. Item 1's is neither. In class we saw a balanced circuit that still glitches.

Build a circuit that is fail-safe and NOT balanced, and show that it still glitches. You may use an activator: a gene whose promoter *requires* a protein to be present rather than being blocked by one.

This one is on the problem set, and it is the one that is actually hard.